

Learning Lab : La Récursivité

Maîtriser l'inconnu en développement Full Stack

Sommaire

1. **Icebreaker** : C'est quoi pour vous ?
2. **Definition** : Le concept d'auto-appel.
3. **Question legitime** : Est-ce toujours pertinent ?
4. **Duel** : Boucles vs Récursivité.
5. **Anatomie** : Stack, Cas de Base, Appel Récursif.
6. **Expertise** : Backtracking et profondeur inconnue.
7. **Workshop** : Mise en pratique.

Question

Selon vous, c'est quoi la recursivite ?

(Repondez par un mot, une image ou une situation du quotidien)

1. Qu'est-ce que la recursivite ?

La recursivite est une methode de resolution de problemes ou une fonction **s'appelle elle-meme** pour resoudre des sous-problemes identiques au probleme initial.

"Pour comprendre la recursivite, il faut d'abord comprendre la recursivite."

2. Exemple simple : Somme recursive

Plutot qu'une boucle `for`, on decompose le probleme.

```
function calculerSommeRec(arr, i = 0) {  
  if (i >= arr.length) return 0; // cas de base  
  return arr[i] + calculerSommeRec(arr, i + 1); // appel recursif  
}
```

3. Est-ce vraiment pertinent ici ?

Pour un tableau, **souvent non**.

Pourquoi :

1. Une boucle fait le job plus simplement.
2. La recursivite consomme la stack (risque de `RangeError` sur tres grands volumes).
3. Avec `slice`, on cree des copies inutiles (cout temps + memoire).

Conclusion pour une liste plate, on prefere `for` ou `reduce` .

3.5 Meme objectif en iteratif

```
function calculerSommeIter(arr) {  
  let total = 0;  
  for (const n of arr) total += n;  
  return total;  
}
```

A retenir

- Iteratif : plus robuste pour les gros tableaux.
- Recursif : utile pour apprendre et pour certains problemes specifiques.

4. Comparaison : Boucles vs Récursivité

Critère	Boucle (For/While)	Récursivité
Approche	Iterative (pas à pas)	Declarative (diviser pour régner)
Données	Listes plates, Tableaux	Arbres, JSON imbriquée, DOM
Complexité	Horizontale (imbrications)	Verticale (profondeur)
Usage	Index connu	Profondeur inconnue

5. Cas concret : explorer des dossiers / sous-dossiers

Exemple classique OS: compter combien de fichiers contient un dossier, quel que soit le nombre de niveaux.

Ici, la profondeur est inconnue: la recursion est plus naturelle qu'une cascade de boucles.

```
import { readdir } from "node:fs/promises";
import path from "node:path";

async function compterFichiers(dossier, entrees = null) {
  if (entrees === null) {
    entrees = await readdir(dossier, { withFileTypes: true });
  }
  if (entrees.length === 0) return 0; // cas de base

  const [entree, ...reste] = entrees;
  const chemin = path.join(dossier, entree.name);
  const courant = entree.isDirectory() ? await compterFichiers(chemin) : 1;

  return courant + (await compterFichiers(dossier, reste));
}
```

5.5 Le vrai gain de la recursivite

Le gain n'est **pas** la performance brute.

Le gain est la **simplicité de modélisation** quand :

1. Les donnees sont en arbre.
2. Le nombre de niveaux est inconnu.
3. On doit explorer, puis revenir en arriere (backtracking).

6. La Stack (Pile d'execution)

L'ordinateur utilise une Stack (LIFO : Last In, First Out) pour gerer les appels.

1. Chaque appel "empile" un nouveau contexte en memoire.
2. L'appel precedent est mis en pause tant que le suivant n'est pas termine.
3. Une fois le cas de base atteint, la pile se "vide" (depilement) pour retourner le resultat final.

6.5 Les deux piliers de la fonction

Pour qu'une fonction recursive soit valide, elle doit respecter deux regles :

1. **Le Cas de Base** : la condition de sortie qui arrete l'auto-appel.
2. **L'Appel Recursif** : l'appel a soi-meme qui doit nous rapprocher du cas de base.

7. Le danger : Stack Overflow

Si la pile devient trop grande (trop d'appels sans condition de sortie), le navigateur sature la memoire.

```
function erreur() {  
  return erreur();  
}  
  
// RangeError: Maximum call stack size exceeded
```

C'est l'equivalent d'une boucle infinie, mais qui sature la memoire vive.

8. Le Backtracking

C'est la puissance de la recursivite : explorer une branche, et si elle echoue (cul-de-sac), revenir en arriere pour explorer la branche suivante.

Usages typiques :

1. Pathfinding (Snake, labyrinthes)
2. Resolution de Sudoku
3. Recherche dans des arborescences complexes

8.5 Exemple : trouver un fichier dans une arborescence

```
async function trouverFichier(dossier, nomFichier, index = 0, entrees = null) {  
  if (entrees === null) {  
    entrees = await readdir(dossier, { withFileTypes: true });  
  }  
  if (index >= entrees.length) return null; // cas de base  
  const entree = entrees[index];  
  const chemin = path.join(dossier, entree.name);  
  if (entree.isFile() && entree.name === nomFichier) {  
    return chemin;  
  }  
  if (entree.isDirectory()) {  
    const trouve = await trouverFichier(chemin, nomFichier);  
    if (trouve) return trouve; // backtracking: on remonte des qu'on a trouve  
  }  
  
  return trouverFichier(dossier, nomFichier, index + 1, entrees);  
}
```


Workshop : Mise en pratique

Objectif

Développer une bibliothèque d'outils capables d'analyser et d'afficher une structure de données dont la profondeur est imprévisible.

Exercices (En continuité)

1. **L'Init** : Créez une fonction `main()` asynchrone pour charger `data.json`.
2. **Le Décompte** : Créez une fonction récursive `lancement(n)` qui affiche un compte à rebours en console avant de traiter les données.
3. **L'Analyseur** : Créez `compterPages(data)` pour retourner le nombre total de nœuds dans l'arbre.
4. **Le Chercheur** : Créez `trouverUrl(data, nom)` qui utilise le backtracking pour trouver l'URL d'une page par son nom.
5. **Le Builder** : Créez `afficherMenu(data, container)` pour générer dynamiquement la structure HTML `<ul>` / `<li>` dans le DOM.

Contraintes

- Chaque exercice doit utiliser une fonction récursive distincte.
- Le code doit fonctionner quel que soit le nombre de niveaux dans le JSON.



alt text